

Computergraphik I

Szenengraphen – am Beispiel der Godot Engine (V 4.7)



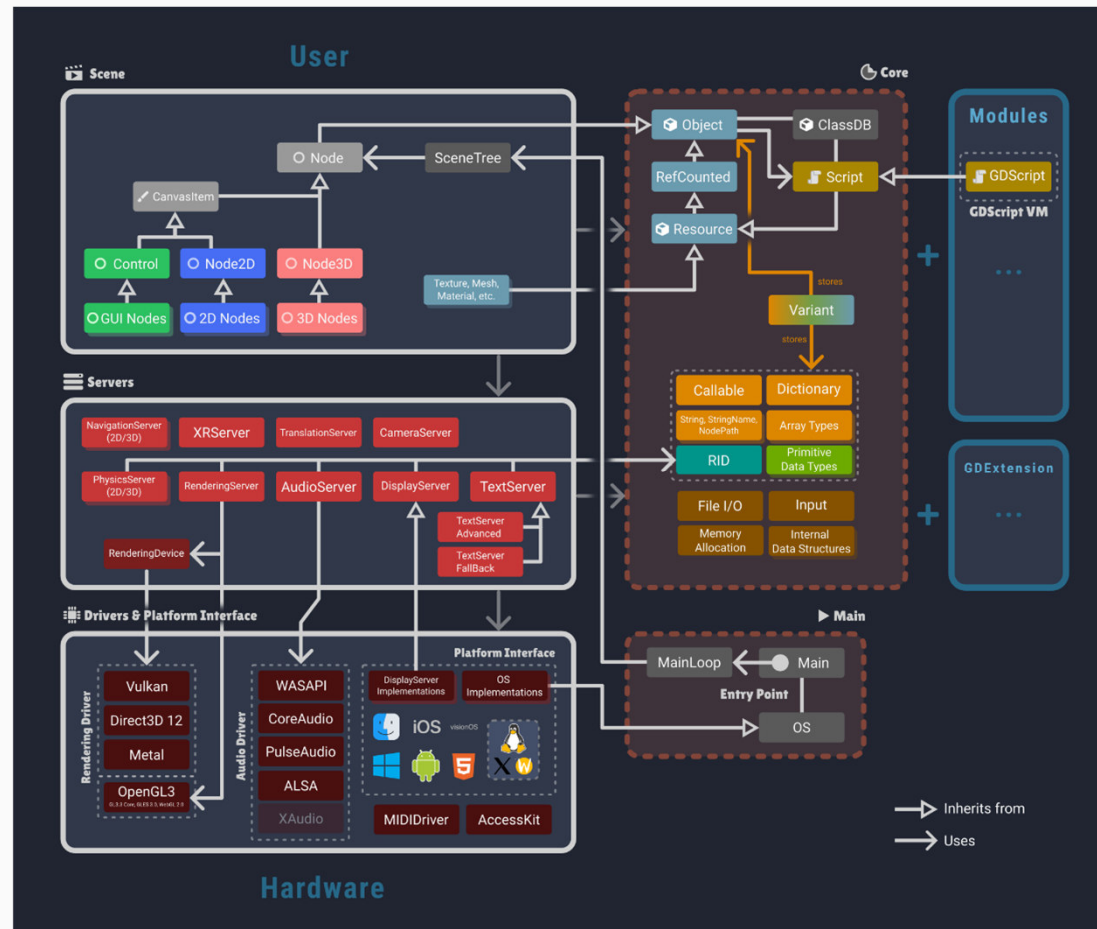
Prof. Dr. Michael Guthe

Sommersemester 2026

AI 5: Graphische Datenverarbeitung
Universität Bayreuth

Godot 4.7 Architektur

- Game Engines sind modular aufgebaut
 - Intern:
 - Core
 - Main
 - Scene
 - Server
 - Driver / Platform

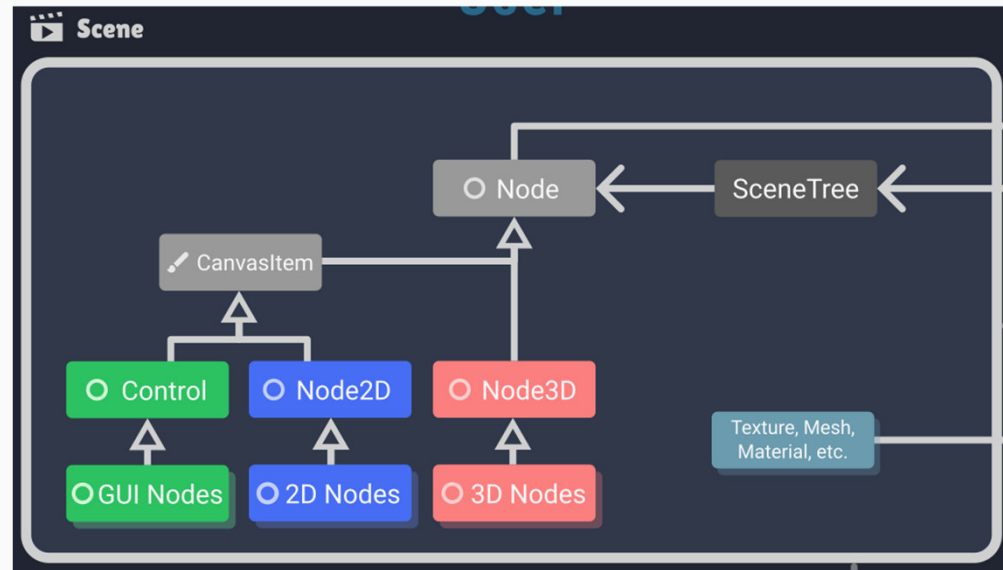


Szenengraphen (1)

- Szene wird in Szenengraphen organisiert

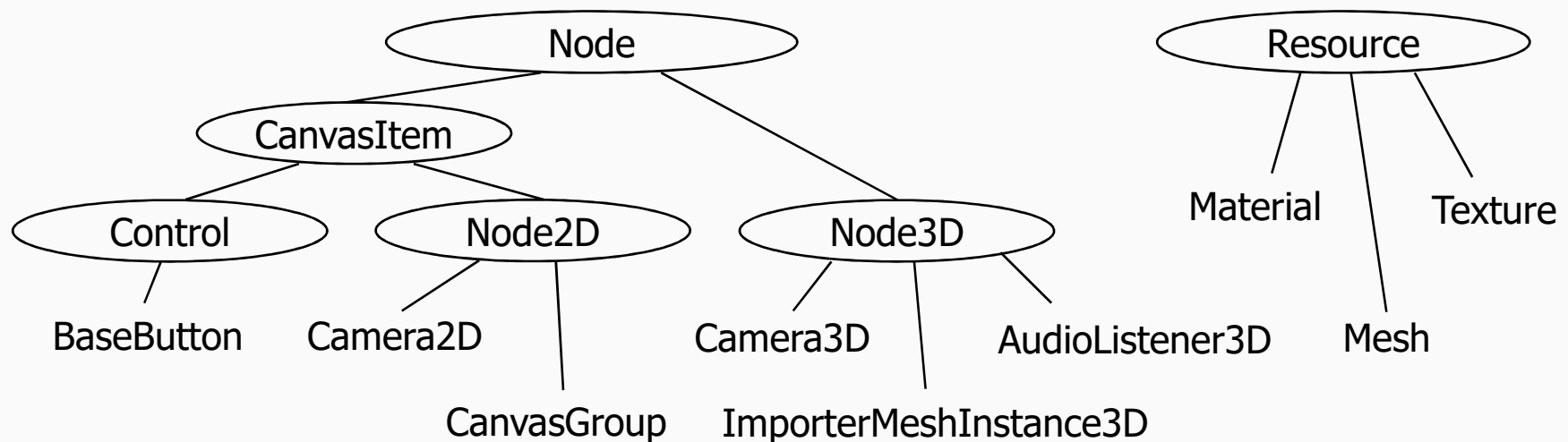
- Motivation:

- Flach vs. Hierarchische Datenstruktur
- Code re-use und Know-how re-use!
- Deskriptiv, nicht imperativ (vgl. C++ vs. C)
 - „Thinking objects ... not rendering processes“



Szenengraphen (2)

- 2 Hierarchien: Szenengraphhierarchie und Klassenhierarchie
- Etliche Klassen sind nicht Teil des Graphen, aber doch Teil der Szene (Resource)



Szenengraphen (3)

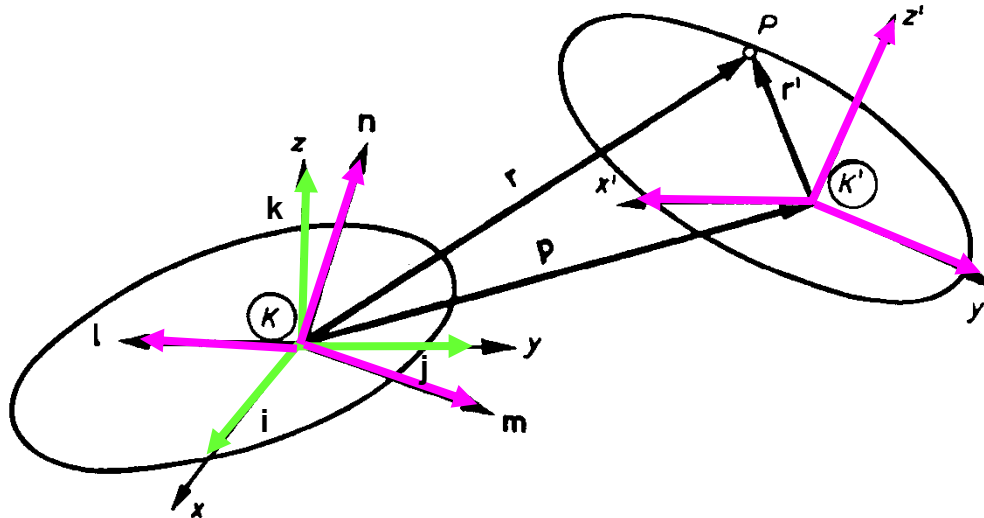
- Semantik der Knoten (Node):
 - Wurzel = „Universum“
 - Blätter = „Inhalt“ (Geometrie, Sound, ...)
 - Innere Knoten = Gruppierung, Transformation, Level-of-Detail, ...
ggf. auch „Inhalt“
- Semantik der Kanten:
 - Transformationsgraph
 - Ein-/Ausschalten von Szenenteilen

Szenengraphen (4)

- Gruppierung: nach welchen Kriterien? Bleibt Applikation überlassen:
 - Geometrische Nähe? (Szenengraph induziert BV-Hierarchie!)
 - Nach Material? (Zustandsänderungen kosten Performance!)
 - Nach logischer Bedeutung? (alle Wasserleitungen, alle Stromkabel, alle Sitze, ...)
 - Nach Transformationskette? (z.B. Planetensystem, Skelett, ...)

Relative Lage starrer Körper (1)

- Seien zwei starre Körper und ihre lokale Koordinatensysteme $\mathbf{K}(\mathbf{i}, \mathbf{j}, \mathbf{k})$ und $\mathbf{K}'(\mathbf{l}, \mathbf{m}, \mathbf{n})$ gegeben
 - Ihre relative Lage wird durch eine affine Transform. beschrieben
 - Die Koordinaten eines Punktes $\mathbf{P}$ im Koordinatensystem $\mathbf{K}$ berechnen sich aus seinen Koordinaten im Koordinatensystem $\mathbf{K}'$ gemäß:



$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} = \underbrace{\begin{bmatrix} \mathbf{l} & \mathbf{m} & \mathbf{n} \end{bmatrix}}_{\text{in Basis } \mathbf{i}, \mathbf{j}, \mathbf{k}} \begin{bmatrix} x' \\ y' \\ z' \end{bmatrix} + \mathbf{p}$$

Relative Lage starrer Körper (2)

- In homogenen 4x4-Matrizen geschrieben ergibt sich:

$$\begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} \mathbf{l} & \mathbf{m} & \mathbf{n} & \mathbf{p} \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} \mathbf{A} & \mathbf{p} \\ \mathbf{0}^T & 1 \end{bmatrix} \begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix}$$

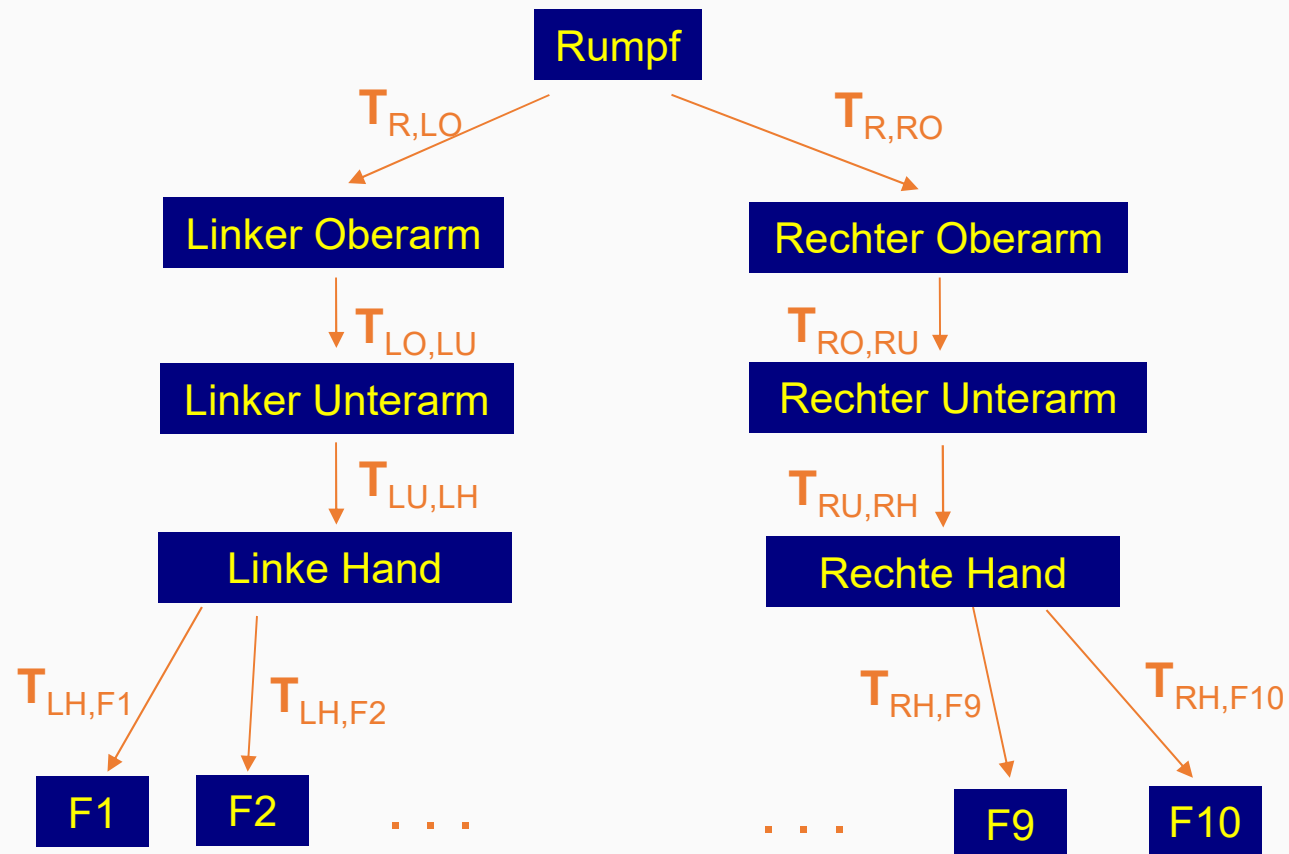
$$\mathbf{T}_{\mathbf{K}\mathbf{K}'} = \begin{bmatrix} \mathbf{A} & \mathbf{p} \\ \mathbf{0}^T & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{l} & \mathbf{m} & \mathbf{n} & \mathbf{p} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Relative Lage starrer Körper (3)

- In komplexeren Szenen bzw. Objekten hängt die Lage von Teilobjekten häufig von anderen Teilobjekten ab, z.B.:
 - Menschmodell: Ringfinger ist Teil der Hand, ihre Lage hängt von der Lage des Unterarms ab, diese wiederum von der Lage des Oberarms, usw.
 - Sonnensystem: die Lage des Mondes hängt von der Lage der Erde ab, deren Lage von der Sonne abhängt, usw.
- Die relative Position und die Abhängigkeit von Objekten untereinander in komplexen Konfigurationen lässt sich einfach durch einen azyklischen **Graphen** darstellen

Relative Lage starrer Körper (4)

- Beispiel: Menschmodell

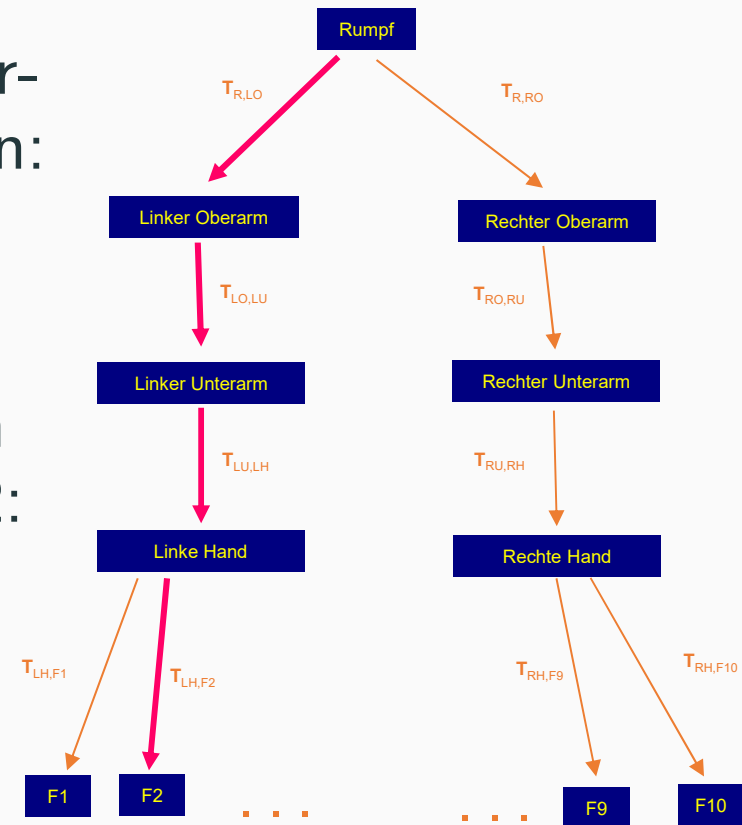
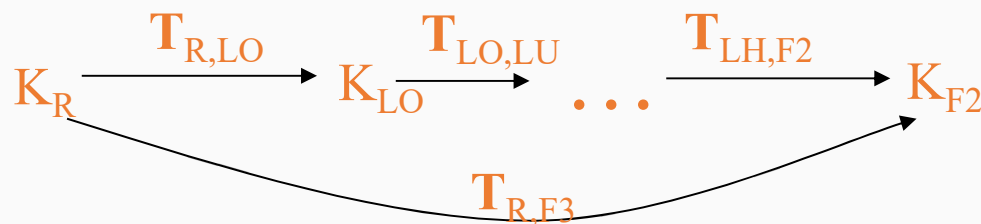


Relative Lage starrer Körper (5)

- Die Transformation zwischen Koordinatensystemen i und j berechnet sich als Hintereinanderausführung der Transformationen:

$$\mathbf{T}_{i,j} = \mathbf{T}_{i,i+1} \cdot \mathbf{T}_{i+1,i+2} \cdot \dots \cdot \mathbf{T}_{j-1,j}$$

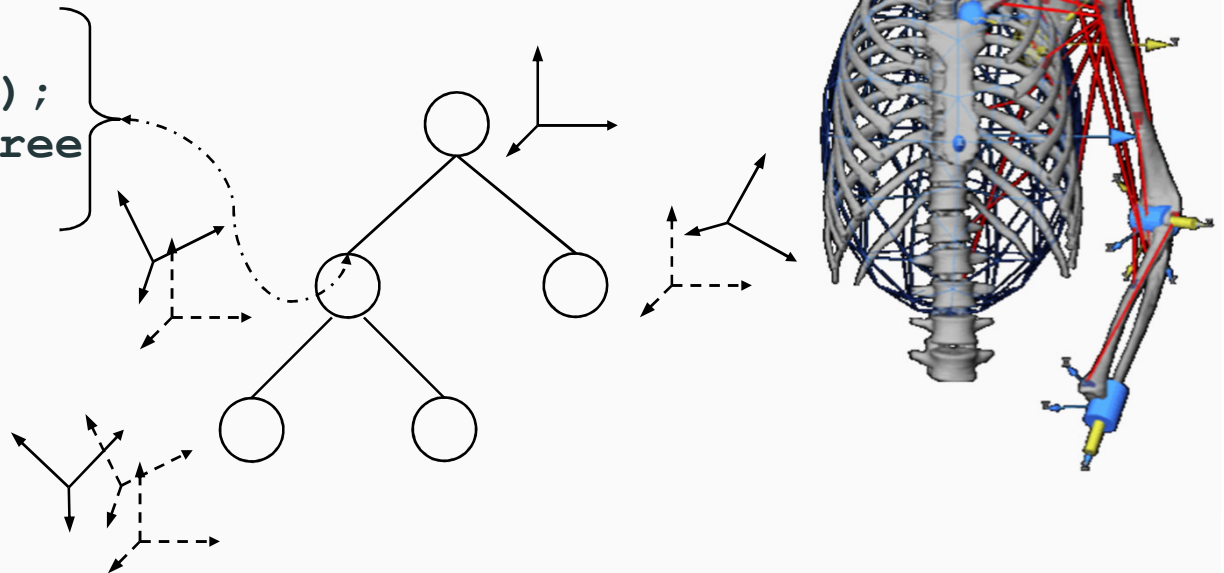
- Die gesamte Transformationskette kann durch Szenengraphen dargestellt werden, z.B. Finger 2:



Relative Lage starrer Körper (6)

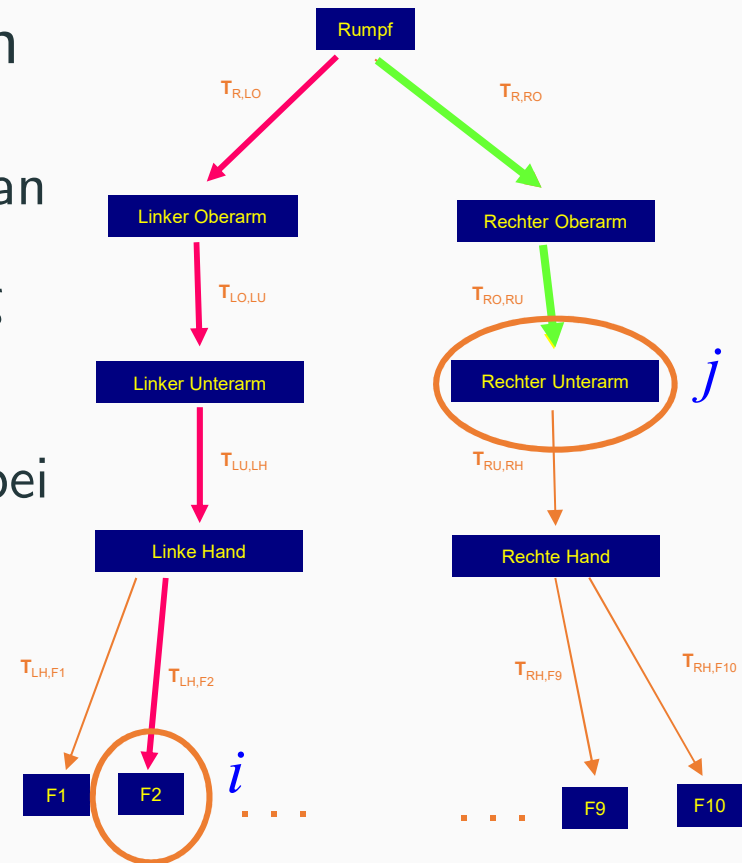
- Transformationssemantik
 - Alle Szenengraphen behandeln diese Semantik gleich
 - Transformationsknoten $\equiv$ neues lokales Koordinatensystem

```
pushMatrix();  
multMatrix(...);  
traverse sub-tree  
popMatrix();
```



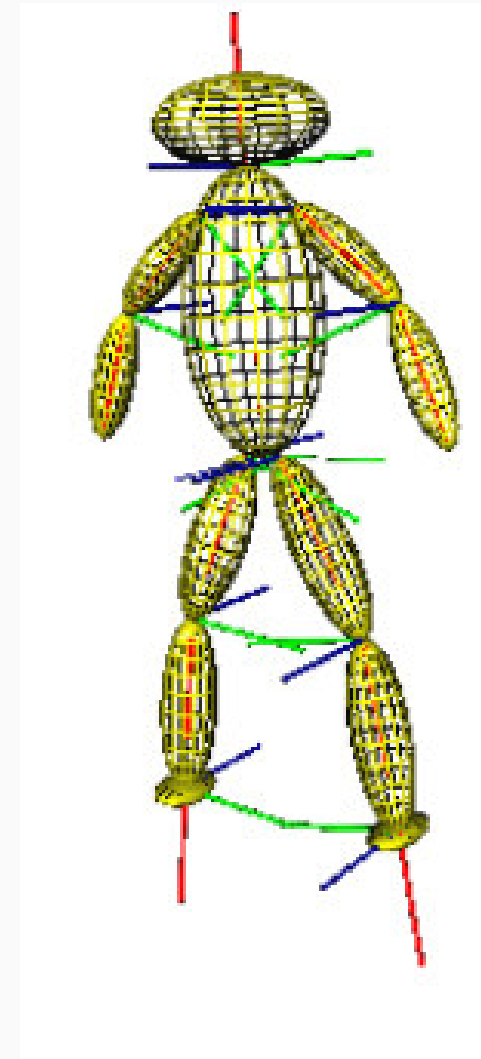
Relative Lage starrer Körper (7)

- Transformation zwischen beliebigen Koordinatensystemen i und j :
 - Die Transformation T_{ij} erhält man durch Multiplikation aller Transformationsmatrizen entlang eines Pfades zwischen i und j
 - Falls die Bewegung entlang der jeweiligen Pfeile erfolgt, wird dabei mit der Transformationsmatrix, sonst mit ihrer Inversen multipliziert
- T_{ij} = Lage von Objekt j im lokalen Koordinatensystems von Objekt i



Relative Lage starrer Körper (8)

- Eine Transformation von Objekt i beeinflusst die Lage von allen Objekten die sich in der Szenengraphhierarchie unterhalb von i befinden
 - Bewegt man z.B. den Rumpf, so folgen die Koordinatensystem aller Teilobjekte dieser Transformation
- Wichtig auch für Animationen



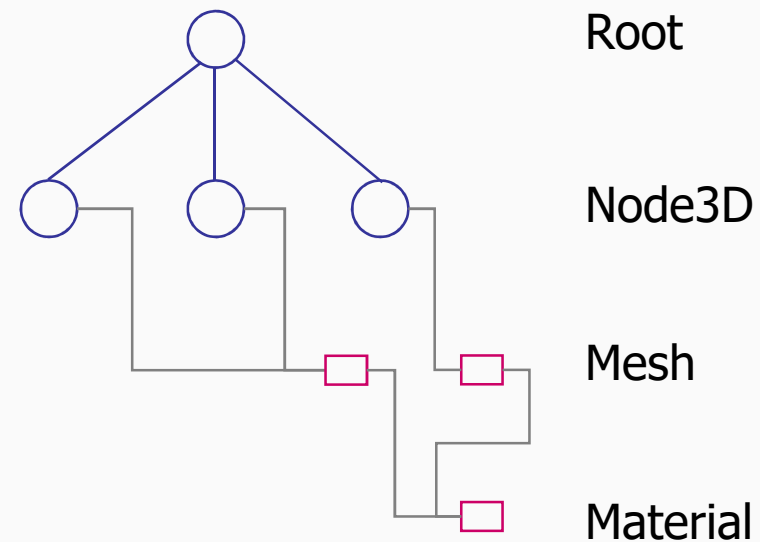
Zustände (1)

- Lichtquellen:
 - Teil des Szenengraphen
 - Semantik:
 - Position/Orientierung von Node
 - Beleuchtet komplette Szene
 - Einschränkungen durch CullMask möglich
- Kamera:
 - Teil des Szenengraphen
 - Kann am Wurzelknoten oder z.B. am Player hängen
 - Mehrere Kameras möglich, aber immer nur eine aktiv



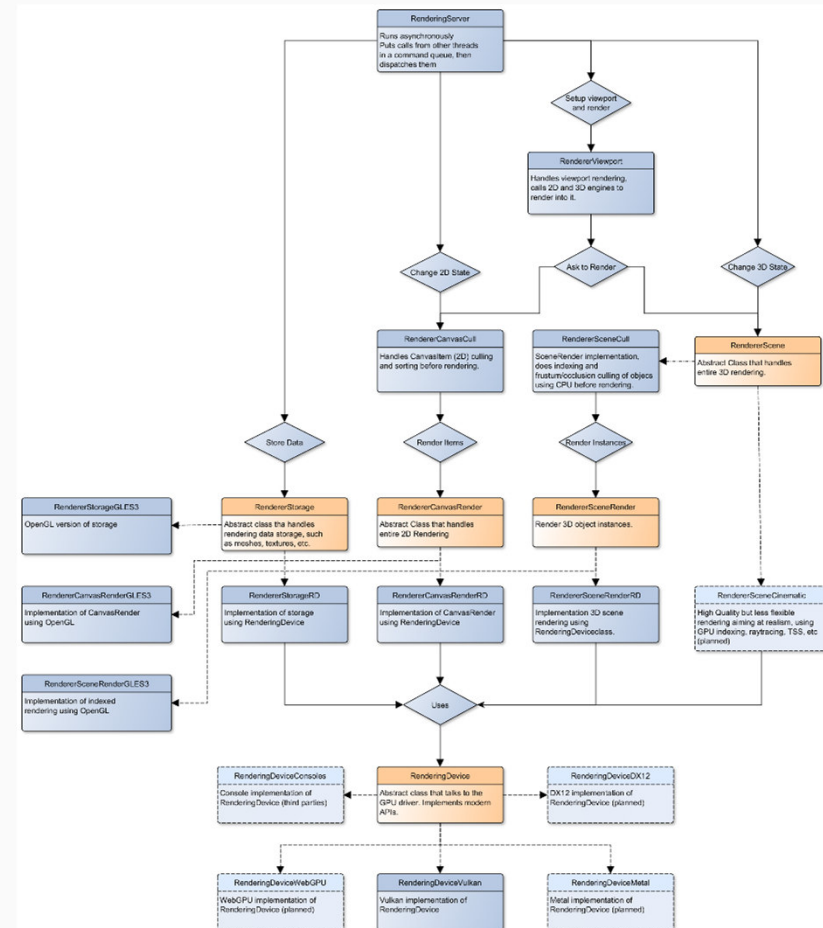
Zustände (2)

- Ressourcen:
 - Eigenschaft eines Knotens oder anderer Ressource
 - Mesh ist Eigenschaft spezieller Knoten
 - Material ist Eigenschaft von Mesh
 - Texture ist Eigenschaft von Material
- Keine Vererbung innerhalb der Szenengraphhierarchie
 - Erleichtert Optimierungen beim Rendern!



Godot 4 interne Renderingarchitektur

- Gesteuert von Rendering Server
 - Eigene Renderer für 2D und 3D
 - Abstraktion von plattformabhängigen APIs
- 3 Modi:
 - Forward+
 - Mobile
 - Compatability

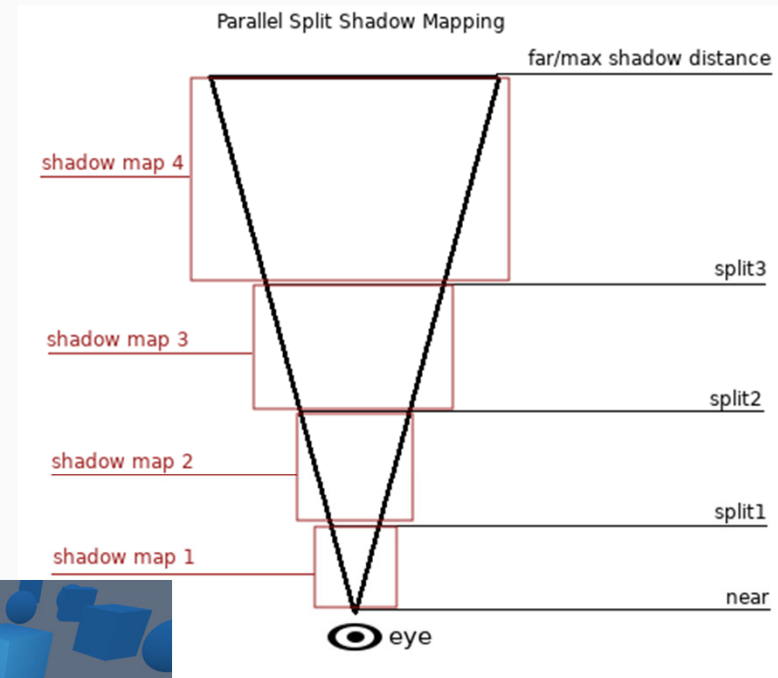
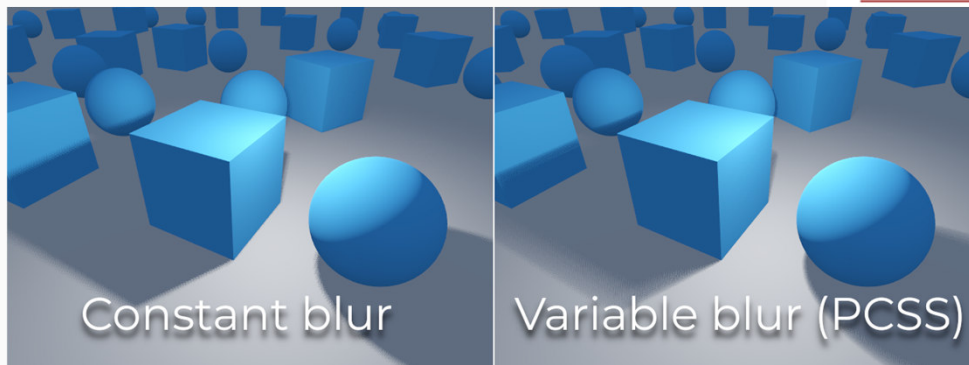


Shading (1)

- Shader ähnlich zu glsl/cgFx
 - Vorimplementierte Funktionen für Schatten usw.
- Beliebige Anzahl an Lichtquellen
 - Speicherung von Lichtquellen in Gitter im Post-Perspective Space (Compute Shader)
 - Dynamische Auswahl von 8 Lichtquellen pro Pixel
- Kein Deferred Shading
 - Würde zu Problemen mit MSAA (adaptivem Anti-Aliasing) führen

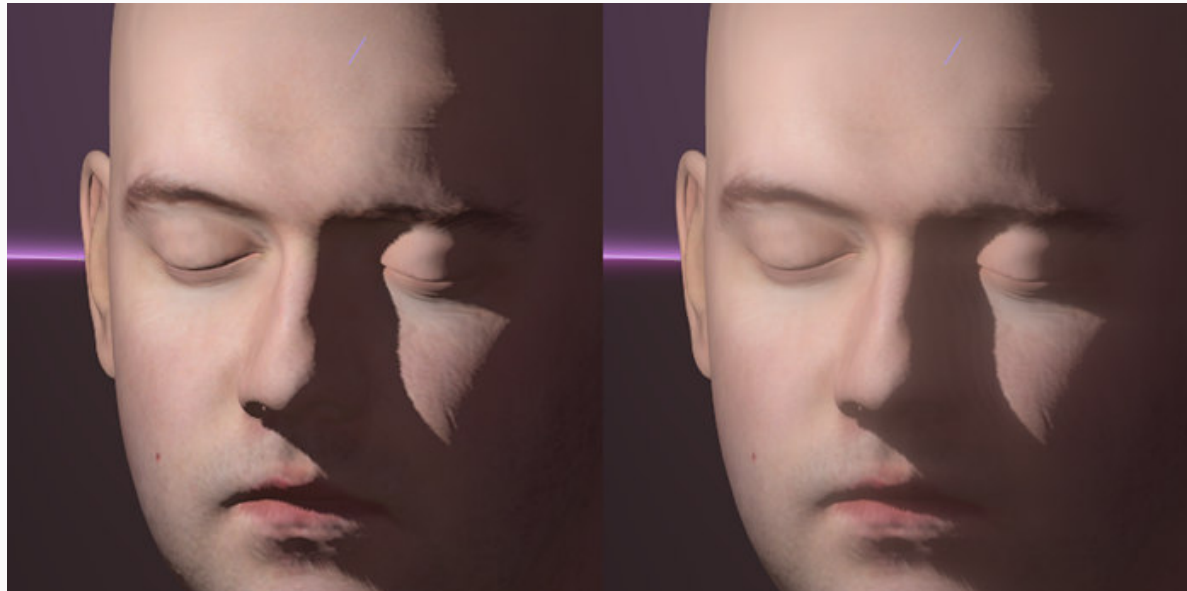
Shading (2)

- Shadow Mapping
 - Cascaded Shadow Maps mit Parallel Split
 - Standard PCF Filter
 - Optional Soft Shadows (PCSS)



Shading (3)

- Subsurface Scattering
 - Screen Space Effekt
 - Filterung der Nachbarpixel
 - Abhängig von 3D Distanz



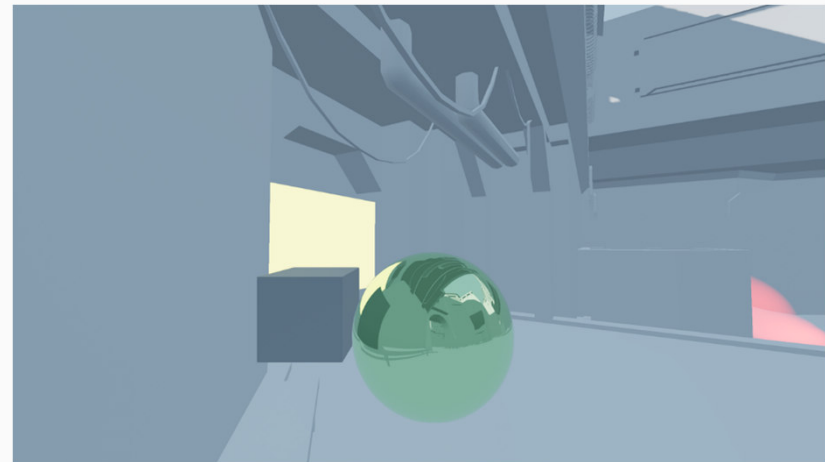
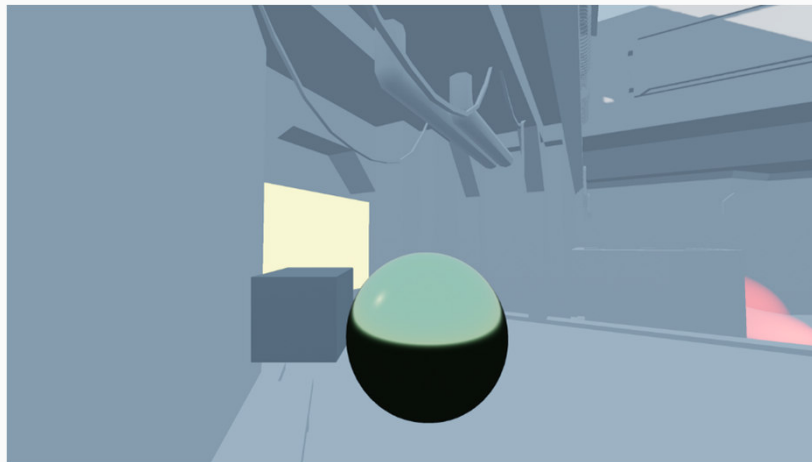
- Occlusion Culling mit hierarchischem z-Buffer
 - Niedrige Auflösung mit Ray Casting erzeugt
 - Test auf der CPU -> Keine Pipeline Stalls
 - Keine Peaks in der Framerate
- Reverse z-Buffer ($z' = 1 - z$)
 - Weniger z-Fighting in der Entfernung
- Level-of-Detail
 - Auswahl im Rendering Server

Globale Beleuchtungseffekte (1)

- Vollständig/teilweise vorberechnete globale Beleuchtung
 - Reflection Probes
 - LightmapGI
 - Screen Space Indirect Light (SSIL)
 - VoxelGI
 - SDFGI

Globale Beleuchtungseffekte (2)

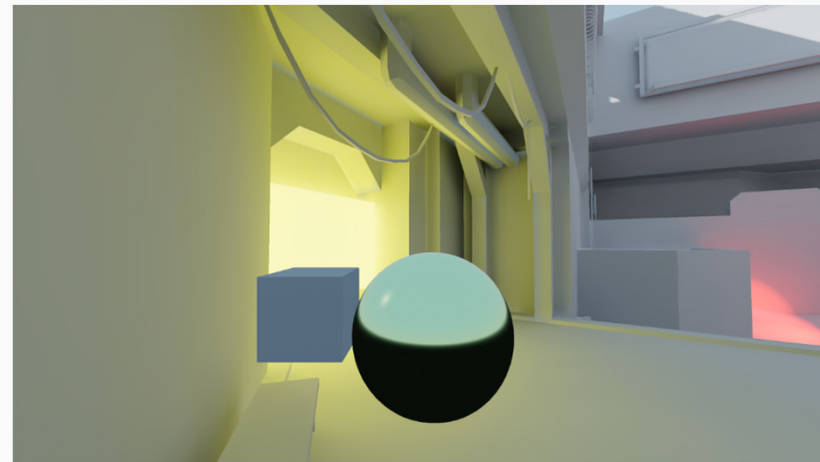
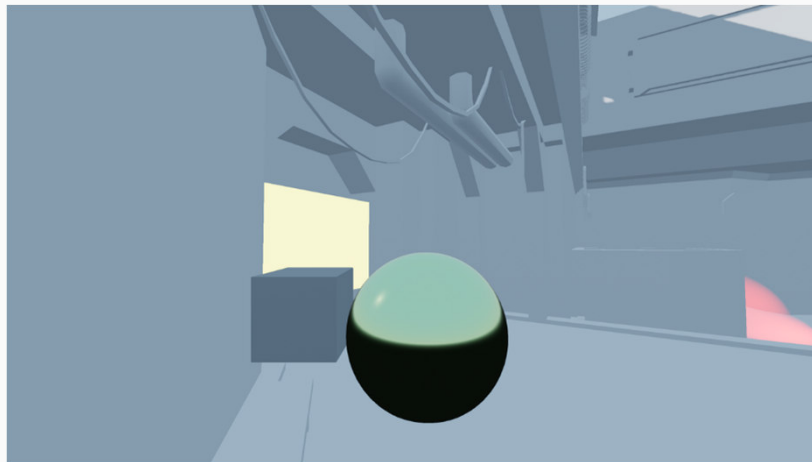
- Reflection Probes
 - Reflection Maps auf regulärem Gitter verteilt



- Kein indirektes Licht
- Können vorberechnet oder zur Laufzeit aktualisiert werden

Globale Beleuchtungseffekte (3)

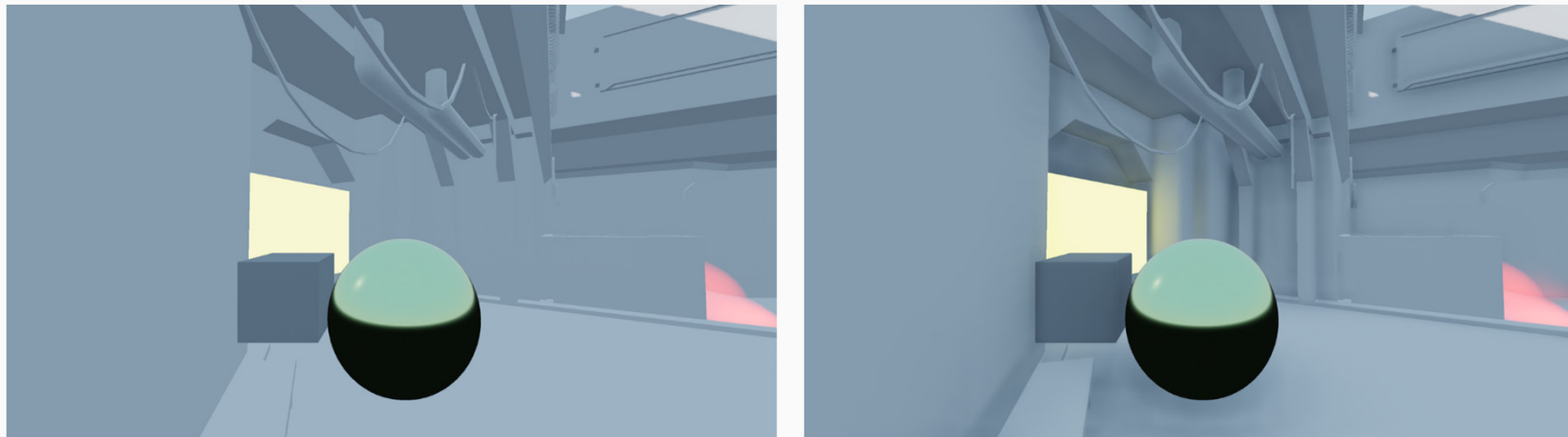
- LightmapGI
 - Vorberechnetes $L_i(x)$, in Texturen gespeichert



- Kann wie L_a nur diffus reflektiert werden
 - Meist Kombination mit Reflection Probes
- Funktioniert nur für statische Umgebungen

Globale Beleuchtungseffekte (4)

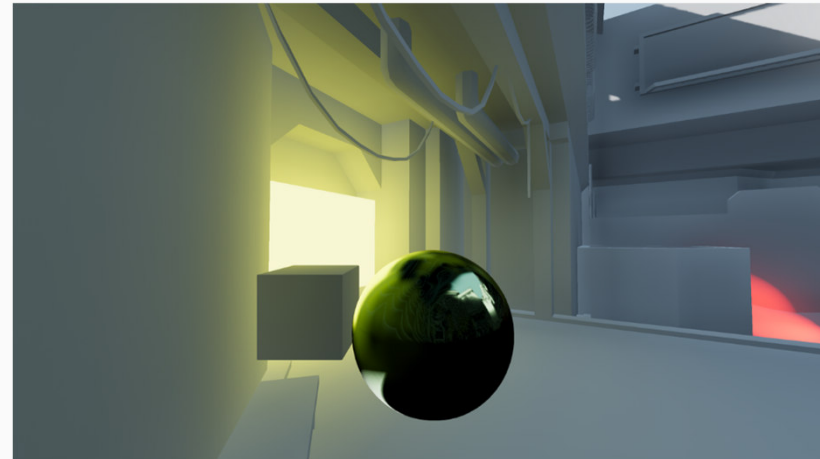
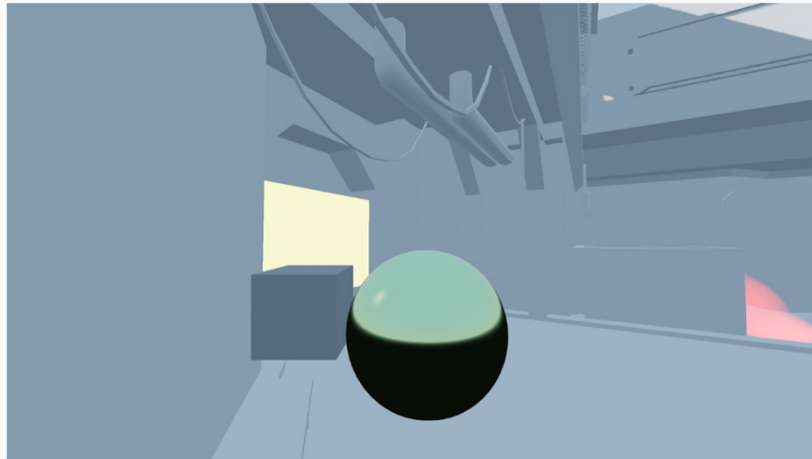
- Screen Space Indirect Light (SSIL)
 - Wie Screen-Space Subsurface Scattering



- Nur diffuse Reflexion
 - Meist Kombination mit Reflection Probes
- Nur erste Reflexion & geringe Distanz zur Lichtquelle

Globale Beleuchtungseffekte (5)

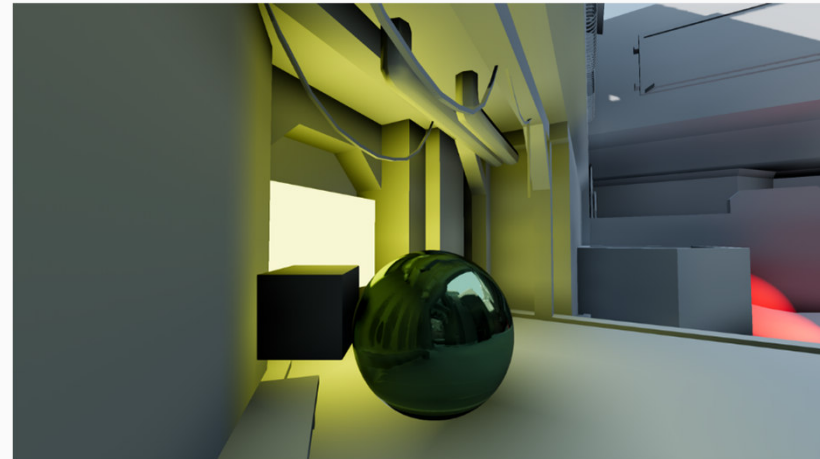
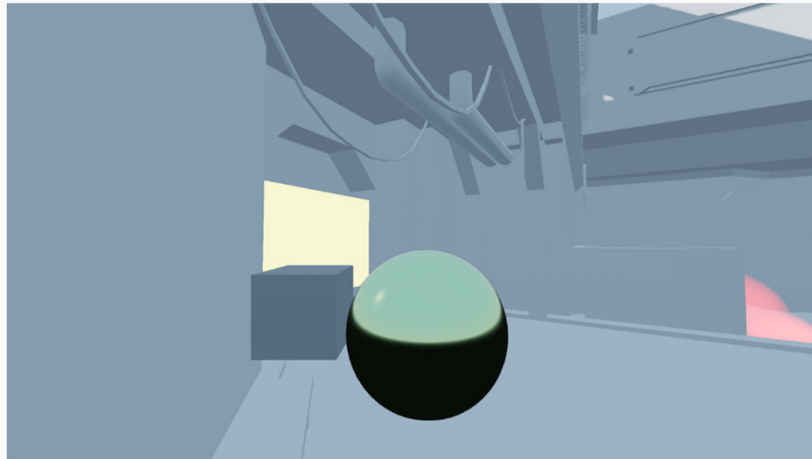
- VoxelGI
 - Vorberechnetes $L_i(\omega_i)$ in Light Probes gespeichert
 - Probes auf regulärem Gitter (Voxel) verteilt



- Nur für statische Umgebungen & begrenzten Bereich
- Globale Beleuchtung durch Ray Tracing (siehe Modul „Globale Beleuchtung“)

Globale Beleuchtungseffekte (6)

- SDFGI
 - Zur Laufzeit berechnetes $L_i(\omega_i)$ in Light Probes
 - Probes in Kaskaden (vgl. Shadow Maps)



- Globale Beleuchtung durch Ray Tracing mit Signed Distance Function (siehe Modul „Globale Beleuchtung“)